

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**

Controle de versão com Git e GitHub

Fundamentos de Git

Aula 3

Código da aula: [DADOS]ANO1C1B3S20A3

Controle de versão
com Git e GitHub

Mapa da Unidade 8 Componente 1

Branches no Git

semana
25

Branches e Merging

semana
27

semana
20

Você está aqui!

Fundamentos de Git

semana
23

Fluxo de trabalho
básico com o Git

semana
29

GitHub



Mapa da Unidade 8 Componente 1

Controle de Versão
com Git e GitHub

Você está aqui!

Fundamentos de Git

20

Aula 3

Código da aula: [DADOS]ANO1C1B3S20A3



Objetivos da aula

- Aprender sobre registro de alterações e repositório remoto.



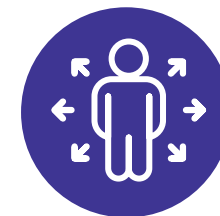
Recursos didáticos

- Recurso audiovisual para exibição de vídeos e imagens.
- Acesso ao laboratório de informática e/ou à internet.



Duração da aula

50 minutos



Competências técnicas

- Usar ferramentas de desenvolvimento de software, como Git e GitHub.



Competências socioemocionais

- Trabalhar em equipes multifuncionais, apoiando colegas, gestores e clientes para alcançar objetivos comuns.

Construindo
o **conceito**

Colaborando via Git

- ▶ Já sabemos que uma das utilidades do Git é viabilizar a colaboração no desenvolvimento de software. No entanto, não é possível trabalhar colaborando na mesma máquina; então, precisamos centralizar o repositório Git em outro lugar na internet!
- ▶ Esse repositório centralizado é o que chamamos de **repositório remoto**.

O que é um repositório remoto?

- ▶ Um repositório remoto é uma versão do seu projeto que está hospedada na internet ou em uma rede, permitindo colaboração entre múltiplos desenvolvedores.
- ▶ Plataformas, como GitHub, GitLab e Bitbucket, facilitam a gestão de repositórios remotos, oferecendo ferramentas para monitorar e controlar alterações no código.

Como funciona um repositório remoto?

- ▶ Para interagir com um repositório remoto, utilizam-se comandos, como *git push*, para enviar alterações locais ao remoto, e *git pull*, para atualizar o repositório local com as mudanças feitas por outros desenvolvedores.
- ▶ Essa colaboração é fundamental para projetos com equipes distribuídas. Outra funcionalidade importante para trabalhar com equipes distribuídas é o registro de alterações.

Explorando o comando *git blame*

- ▶ O comando *git blame* é usado para rastrear quem fez alterações em cada linha de um arquivo, exibindo o autor, a data da mudança e o *commit* relacionado.
- ▶ Isso é extremamente útil para entender mudanças específicas e solucionar problemas relacionados a alterações no código. Também pode ser relevante para pedir ajuda ao autor de um trecho do código que precisamos evoluir.

Construindo
o **conceito**

Utilizando *git blame* para análise de código

- ▶ Se você encontrar um bug em uma seção do código, pode usar *git blame* para identificar rapidamente quem contribuiu para aquele trecho e entender melhor o contexto das mudanças para facilitar a correção.
- ▶ Se você, por outro lado, for evoluir uma lógica específica da área de negócio da sua empresa, pode perguntar para o desenvolvedor anterior por que ele implementou alguma *feature* de certa maneira.

Colocando
em **prática**

Mapa mental de Git

1. Reúnam-se em grupos de quatro pessoas.
2. Construam um **mapa mental com os comandos do Git** que vocês já conhecem.
3. No final da atividade, coloque o mapa mental elaborado em uma apresentação do PowerPoint.



Até o final da aula



Quatro pessoas

Ser
sempre +

Situação

Em grupos de até quatro alunos e utilizando os aprendizados desta semana, reflita sobre o problema a seguir:

Um erro em produção ocorreu na aplicação que você desenvolveu. O erro é significativo e, logo, o time se reúne para fazer uma *War Room* e produzir um *Fix*.

Você encontra o erro e utiliza o comando *git blame* para entender quando foram subidas as últimas alterações no arquivo. Quando faz isso, você descobre que um amigo seu fez as últimas mudanças por acidente, sendo responsável pelo erro.

O que você faz nessa situação? Discuta em grupos.



© Getty Images

O que nós
**aprendemos
hoje?**

Então, ficamos assim...

- 1** Discutimos o conceito e a funcionalidade dos repositórios remotos, essenciais para a colaboração em projetos de desenvolvimento de software.
- 2** Exploramos o comando *git blame*, uma ferramenta valiosa para identificar a autoria de alterações específicas em arquivos, facilitando o rastreamento de contribuições individuais e ajudando a solucionar questões relacionadas a bugs ou mudanças problemáticas no código.
- 3** Destacamos a importância de não usar o *git blame* como uma ferramenta de acusação, uma vez que isso é contraprodutivo em comparação ao trabalho em equipe.

Saiba mais

Quer aprender a criar um repositório remoto?

Este artigo da Alura pode te ajudar!

D' ROMERO, L. L. Criando um repositório remoto em GitHub. *Alura*, 9 mar. 2023. Disponível em:

<https://www.alura.com.br/artigos/criando-repositorio-remoto-github/>. Acesso em: 30 maio 2024.

Referências da aula

BELL, P. ; BEER, B. *Introdução ao GitHub*: um guia que não é técnico. São Paulo: Novatec, 2014.

CHACON, S; STRAUB, B. *Pro Git*. Apress, 2024. Disponível em: <https://git-scm.com/book/pt-br/v2/>. Acesso em: 30 maio 2024.

D' ROMERO, L. L. Criando um repositório remoto em GitHub. *Alura*, 9 mar. 2023. Disponível em: <https://www.alura.com.br/artigos/criando-repositorio-remoto-github/>. Acesso em: 30 maio 2024.

GITHUB DOCS. *Documentação de introdução ao github*, [s.d.]. Disponível em: <https://docs.github.com/pt/get-started/>. Acesso em: 30 maio 2024.

GITHUB. *Página inicial*, [s.d.]. Disponível em: <https://github.com/>. Acesso em: 31 maio 2024.

GITLAB. *Página inicial*, [s.d.]. Disponível em: <https://about.gitlab.com/>. Acesso em: 31 maio 2024.

Identidade visual: imagens © Getty Images.

Educação Profissional Paulista

Técnico em
**Ciência de
Dados**